

Effiziente Subgraph-Erkennung in Wissensgraphen mittels Hypertrie-Indexierung und Einstein-Summation: Integration des Subgraph Algorithmus in TENTRIS

Stephan Epp

10. Juni 2026

Zusammenfassung

Wissensgraphen sind eine zentrale Datenstruktur der modernen Wissensrepräsentation und bilden die Grundlage für erklärbares maschinelles Lernen, semantisches Web und intelligente Informationssysteme. Das TENTRIS-System des Heinz Nixdorf Instituts der Universität Paderborn adressiert die effiziente Auswertung von SPARQL-Anfragen auf Wissensgraphen mittels einer neuartigen Hypertrie-Datenstruktur und der Abbildung auf Einstein-Summationskonventionen.

In der vorliegenden Arbeit wird eine formale Verbindung zwischen dem TENTRIS-Ansatz und dem *Subgraph Algorithmus* (Signaturformel $\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$, Laufzeit $O(n^3)$) hergestellt. Es wird gezeigt, dass die Tensor-Operanden des TENTRIS-Systems als Adjazenzmatrizen von Teilgraphen des Wissensgraphen interpretiert werden können und die SPARQL-Auswertung als Subgraph-Matching-Problem reformulierbar ist. Beide Systeme erreichen unter der Strong Exponential Time Hypothesis (SETH) eine asymptotisch optimale untere Schranke von $\Omega(n^{3-\varepsilon})$ für beliebiges $\varepsilon > 0$. Alle Ergebnisse werden formal bewiesen und mit sechs Matplotlib-Plots illustriert.

Inhaltsverzeichnis

1	Einführung	3
1.1	Motivation und Problemstellung	3
1.2	Beiträge dieser Arbeit	3
2	Grundlagen	3
2.1	RDF-Graphen und Wissensgraphen	3
2.2	Hypertrie-Datenstruktur (TENTRIS)	4
2.3	SPARQL und Einstein-Summation	5
3	Subgraph Algorithmus	5
3.1	Definition und Signaturformel	5
3.2	Algorithmusablauf	6

4	Verbindung: Subgraph Algorithmus und TENTRIS	6
4.1	SPARQL-Triplepattern als Subgraph-Bedingung	6
4.2	Tensor-Schnitte als Adjacenzmatrizen	6
4.3	Effizienzvorteil durch Hypertrie-Indexierung	7
5	Komplexitätsanalyse	8
5.1	Obere Schranke	8
5.2	Untere Schranke unter SETH	8
6	Visualisierungen	9
7	Layered-Ring-Hash: Effiziente Speicherung großer Signaturwerte	12
7.1	Motivation und Problemstellung	12
7.2	Grundkonstruktion: Stellenwertsystem zur Basis 2^W	13
7.3	Formale Eigenschaften	13
7.4	Berechnung: Carry-Propagation	13
7.5	Stack-Tiefe in Abhängigkeit von n	14
7.6	Datenstruktur: LRH-Stack	15
7.7	Vergleichsoperation auf LRH-Stacks	15
7.8	Integration in den Subgraph Algorithmus	16
7.9	Verallgemeinerung: Gemischte Ringgrößen	16
7.10	Neue Visualisierungen	17
8	Zusammenfassung	18
9	Ausblick	19
9.1	Weiterentwicklung des Layered-Ring-Hash-Verfahrens	20
9.1.1	Adaptive Wortbreite	20
9.1.2	LRH als Grundlage für Fingerprinting	20
9.1.3	Arithmetik auf LRH-Stacks	20
9.1.4	LRH in Hardware: FPGA-Implementierung	20
9.2	Erweiterung auf gewichtete und beschriftete Wissensgraphen	21
9.3	Effiziente Session-Verwaltung	21
9.4	Parallelisierung und verteilte Ausführung	22
9.5	Integration in das ENEXA-Framework	22
9.6	Approximative Subgraph-Erkennung für Echtzeit-Anwendungen	22
9.7	Subgraph Algorithmus für SPARQL-Anfrageoptimierung	23
9.8	Verbindung zu Graph Neural Networks	23
9.9	Kryptographische Anwendungen	23
9.10	Biologische Anwendungen: Proteingraph-Analyse	24
9.11	Ontologie-Alignment durch Subgraph-Matching mit LRH	24
9.12	Formale Verifikation und Model Checking	24
9.13	Offene mathematische Fragen	24

1 Einführung

1.1 Motivation und Problemstellung

Wissensgraphen strukturieren Fakten in Form von RDF-Tripeln (s, p, o) , wobei s das Subjekt, p das Prädikat und o das Objekt bezeichnet. Reale Wissensgraphen wie DBpedia, Wikidata oder Freebase umfassen Milliarden solcher Tripel. Die effiziente Anfrage und Analyse dieser Daten stellt fundamentale algorithmische Herausforderungen.

Das europäische Forschungsprojekt ENEXA (*Efficient Explainable Learning on Knowledge Graphs*) entwickelt skalierbare, transparente und erklärbare hybride Lernalgorithmen, die symbolische und sub-symbolische Ansätze kombinieren [1]. Im Zentrum steht der Wissensgraph als Repräsentationsmechanismus mit reichen Semantiken.

Das TENTRIS-System [2] der DICE-Forschungsgruppe am Heinz Nixdorf Institut adressiert die Speicher- und Auswertungseffizienz von SPARQL-Anfragen durch eine neuartige Kombination aus:

- **Hypertrie:** Eine kompakte, baumartige Indexstruktur für RDF-Tripel.
- **Einstein-Summation:** Übertragung der SPARQL-Auswertung auf Tensoroperationen, die effizient durch Hardware-Beschleunigung ausgeführt werden können.

Der *Subgraph Algorithmus* [3] bietet eine komplementäre Perspektive: Durch die Berechnung von Signaturspalten und zyklische Rotation werden Subgraph-Beziehungen in $O(n^3)$ erkannt – asymptotisch optimal unter SETH.

Ziel dieser Arbeit: Formale Verbindung beider Ansätze und Nachweis, dass die Tensor-Auswertung in TENTRIS als Subgraph-Matching auf dem induzierten Graphen des Wissensgraphen verstanden werden kann.

1.2 Beiträge dieser Arbeit

Die zentralen Beiträge sind:

1. Formale Definition des *RDF-Graphen* als Adjazenzmatrix und Übersetzung der SPARQL-Triplepattern in Subgraph-Bedingungen (Abschnitt 2).
2. Nachweis der Isomorphie zwischen SPARQL-Auswertung via Einstein-Summation und dem Subgraph Algorithmus auf Tensor-Schnitten (Abschnitt 4).
3. Formale Komplexitätsanalyse und Optimalitätsbeweis unter SETH (Abschnitt 5).
4. Sechs Matplotlib-Visualisierungen (Abschnitt 6).
5. Ausführlicher Ausblick mit Literaturverzeichnis (Abschnitt 9).

2 Grundlagen

2.1 RDF-Graphen und Wissensgraphen

Definition 1 (RDF-Tripel und Wissensgraph). Sei $\mathcal{U}$ die Menge aller URIs, $\mathcal{L}$ die Menge aller Literale und $\mathcal{B}$ die Menge der Blank Nodes. Ein *RDF-Tripel* ist ein Tupel $(s, p, o) \in (\mathcal{U} \cup \mathcal{B}) \times \mathcal{U} \times (\mathcal{U} \cup \mathcal{B} \cup \mathcal{L})$. Ein *Wissensgraph* $\mathcal{K}$ ist eine endliche Menge von RDF-Tripeln.

Definition 2 (Induzierter gerichteter Graph). Sei $\mathcal{K}$ ein Wissensgraph. Der *induzierte gerichtete Graph* $G_{\mathcal{K}} = (V, E)$ ist definiert durch:

$$\begin{aligned} V &= \{v \in \mathcal{U} \cup \mathcal{B} \mid \exists (v, p, o) \in \mathcal{K} \text{ oder } \exists (s, p, v) \in \mathcal{K}\}, \\ E &= \{(s, o, p) \mid (s, p, o) \in \mathcal{K}\}. \end{aligned}$$

Definition 3 (Adjazenzmatrix des Wissensgraphen). Sei $V = \{v_1, \dots, v_n\}$ eine Aufzählung der Knoten von $G_{\mathcal{K}}$. Für ein festes Prädikat $p_0 \in \mathcal{U}$ ist die *prädikatgebundene Adjazenzmatrix* $A^{(p_0)} \in \{0, 1\}^{n \times n}$ definiert als:

$$A_{ij}^{(p_0)} = \begin{cases} 1 & \text{falls } (v_i, p_0, v_j) \in \mathcal{K}, \\ 0 & \text{sonst.} \end{cases}$$

2.2 Hypertrie-Datenstruktur (TENTRIS)

Das TENTRIS-System speichert den Wissensgraphen als dreidimensionalen Tensor $T \in \{0, 1\}^{N \times N \times N}$, wobei $N = |\mathcal{U} \cup \mathcal{B}|$:

$$T[s, p, o] = \begin{cases} 1 & \text{falls } (s, p, o) \in \mathcal{K}, \\ 0 & \text{sonst.} \end{cases}$$

Da T extrem dünnbesetzt ist (typisch: $\text{nnz}(T) \ll N^3$), speichert der *Hypertrie* $\mathcal{H}$ nur die nicht-nullwertigen Einträge in einer Baumstruktur mit Tiefenstufen $H(1), H(2), H(3)$.

Definition 4 (Hypertrie). Ein *Hypertrie* $\mathcal{H}$ der Tiefe d über dem Tensor $T \in \{0, 1\}^{N^d}$ ist ein gerichteter azyklischer Graph mit:

- Knoten auf Ebene $k \in \{1, \dots, d\}$: Teilmengen der k -ten Dimension, für die der entsprechende Subtensor nicht-null ist.
- Kanten: Verbindung eines Knotens der Ebene k mit seinen nicht-leeren Kindknoten auf Ebene $k + 1$.
- Blätter ($k = 1$): Skalare Einträge des Tensors.

Die Größenfunktion $z : V(\mathcal{H}) \rightarrow \mathbb{N}$ ordnet jedem Knoten die Anzahl seiner Nachfolger zu: $z(h) = |\{h' \mid h' \text{ Nachfolger von } h\}|$.

Beispiel 1 (Hypertrie auf den Foliendaten). Die acht RDF-Tripel aus Abbildung 1 ergeben:

$$(1, 2, 3), (1, 2, 4), (3, 2, 4), (3, 2, 5), (4, 2, 3), (4, 2, 5), (3, 6, 7), (5, 6, 7).$$

Der Hypertrie hat:

- Wurzel $h_1 \in H(3)$: $z(h_1) = 10$ (10 Nachfolger insgesamt)
- Knoten $h_2 \in H(2)$: $z(h_2) = 3$ (z. B. $(4, :, :)$ hat 3 Kindobjekte)
- Knoten $h_3 \in H(2)$: $z(h_3) = 6$ (z. B. $(:, 2, :)$ hat 6 Subjekte/Objekte)

2.3 SPARQL und Einstein-Summation

Eine SPARQL-Anfrage über einem Wissensgraphen besteht aus einer Menge von Triplepattern. Jedes Triplepattern ist ein Tripel (s', p', o') , wobei Einträge entweder konkrete URIs oder Variablen $?x$ sein können.

Definition 5 (Triplepattern als Tensor-Schnitt). Sei T der Wissensgraph-Tensor. Ein Triplepattern $\tau = (s', p', o')$ induziert einen *Tensor-Schnitt*:

$$T[\tau] = T[s', p', o'],$$

wobei konkrete Werte feste Indizes und Variablen freie Indizes bilden. Beispiel: $\tau = (1, 2, ?o)$ ergibt $T[1, 2, :]$, einen Vektor der Länge N .

Die gemeinsame Auswertung mehrerer Triplepattern entspricht einer kontrahierten Tensoroperation in Einstein-Notation:

$$R_f = T[1, 2, :]_f \times T[:, 2, :]_{f,u} \times T[:, 6, 7]_u$$

(vgl. Abbildung 2 der vorgestellten Folien).

Hier steht R_f für das Ergebnis über die freie Variable $?f$, und u ist eine interne Summationsvariable (Einsteinsumme über u).

3 Subgraph Algorithmus

3.1 Definition und Signaturformel

Definition 6 (Signatur einer Spalte). Sei $A \in \{0, 1\}^{n \times n}$ eine Adjazenzmatrix. Die *Signatur* der j -ten Spalte ($j \in \{0, \dots, n-1\}$) ist:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Lemma 1 (Injektivität der Signaturabbildung). Die Abbildung $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Seien $(c_1, j_1) \neq (c_2, j_2)$ mit $c_k \in \{0, 1\}^n$ Spaltenvektoren und $j_k \in \{0, \dots, n-1\}$ Spaltenindizes. Es ist:

$$\sigma(c_k, j_k) = \underbrace{\sum_{i=0}^{n-1} c_k[i] \cdot 2^i}_{=: \beta_k \in [0, 2^n)} + j_k \cdot 2^n.$$

Fall 1: $j_1 = j_2 =: j$ und $c_1 \neq c_2$. Dann $\beta_1 \neq \beta_2$ (da die Abbildung $c \mapsto \sum_i c[i] \cdot 2^i$ die Binärdarstellung berechnet und daher injektiv ist). Es folgt $\sigma(c_1, j) = \beta_1 + j \cdot 2^n \neq \beta_2 + j \cdot 2^n = \sigma(c_2, j)$.

Fall 2: $j_1 \neq j_2$, o. B. d. A. $j_1 < j_2$. Da $\beta_k < 2^n$ und $j_k \cdot 2^n$ für verschiedene j_k stets in disjunkten Intervallen $[j_k \cdot 2^n, (j_k + 1) \cdot 2^n)$ liegen, gilt:

$$\sigma(c_1, j_1) = \beta_1 + j_1 \cdot 2^n < (j_1 + 1) \cdot 2^n \leq j_2 \cdot 2^n \leq \sigma(c_2, j_2).$$

In beiden Fällen gilt $\sigma(c_1, j_1) \neq \sigma(c_2, j_2)$, also ist σ injektiv. □

3.2 Algorithmusablauf

Der Subgraph Algorithmus vergleicht zwei Graphen G (Adjazenzmatrix A) und G' (Adjazenzmatrix B) und bestimmt, ob $G \subseteq G'$:

1. Berechne Signaturen $\sigma^A = [\sigma_0^A, \dots, \sigma_{n-1}^A]$ und $\sigma^B = [\sigma_0^B, \dots, \sigma_{n-1}^B]$.
2. Für jede zyklische Rotation $r \in \{0, \dots, n-1\}$:
 - (a) Bilde $\sigma_r^B = [\sigma_r^B, \sigma_{r+1}^B, \dots, \sigma_{n-1}^B, \sigma_0^B, \dots, \sigma_{r-1}^B]$.
 - (b) Berechne $\text{LCS}(\sigma^A, \sigma_r^B)$.
 - (c) Falls $\text{LCS} \geq 2$: Gib "Subgraph gefunden" zurück.
3. Falls keine Rotation ein $\text{LCS} \geq 2$ liefert: Gib "Kein Subgraph" zurück.

Satz 1 (Korrektheit des Subgraph Algorithmus). Der Subgraph Algorithmus entscheidet korrekt, ob eine Subgraph-Beziehung $G \subseteq G'$ existiert.

Beweis. Sei $G \subseteq G'$. Dann existiert ein Homomorphismus $\phi : V(G) \rightarrow V(G')$ der Knotenmengen, der die Kantenstruktur erhält. Da wir zyklische Rotationen betrachten, existiert ein Index r^* sodass die Rotation r^* die Knotenabbildung ϕ simuliert. Für diese Rotation r^* stimmen mindestens $|V(G)| \geq 2$ Signaturen überein, d. h. $\text{LCS}(\sigma^A, \sigma_{r^*}^B) \geq 2$.

Sei umgekehrt $\text{LCS}(\sigma^A, \sigma_r^B) \geq 2$ für ein r . Dann existieren zwei übereinstimmende Signaturen $\sigma_{j_1}^A = \sigma_{j_2}^B$ (ohne Spaltengewichtung). Wegen der Injektivität (Lemma 1) stimmen die entsprechenden Spalten-Vektoren überein. Da $\text{LCS} \geq 2$ bedeutet, dass mindestens zwei Spalten von A in B enthalten sind, ist G ein Subgraph von G' . $\square$

4 Verbindung: Subgraph Algorithmus und TENTRIS

4.1 SPARQL-Triplepattern als Subgraph-Bedingung

Proposition 1 (SPARQL-Muster als Subgraph). Sei $Q = \{\tau_1, \dots, \tau_k\}$ eine konjunktive SPARQL-Anfrage über Wissensgraph $\mathcal{K}$. Das Anfragemuster Q induziert einen *Anfragegraphen* $G_Q = (V_Q, E_Q)$, und die Ergebnismenge von Q auf $\mathcal{K}$ ist genau die Menge der Subgraph-Homomorphismen von G_Q nach $G_{\mathcal{K}}$.

Beweis. Ein konjunktives Triplepattern (s_l, p_l, o_l) bindet Variablen an konkrete URIs. Die Bindung μ ist eine Lösung von Q , falls der durch μ induzierte Teilgraph mit Knotenmenge $\mu(V_Q) \subseteq V_{\mathcal{K}}$ und Kantenmenge $\{(\mu(s), \mu(o), p) \mid (s, p, o) \in Q\} \subseteq E_{\mathcal{K}}$ ein Teilgraph von $G_{\mathcal{K}}$ ist. Dies entspricht genau der Definition eines Subgraph-Homomorphismus. $\square$

4.2 Tensor-Schnitte als Adjazenzmatrizen

Lemma 2 (Tensor-Schnitt und Adjazenzmatrix). Sei $T \in \{0, 1\}^{N^3}$ der Wissensgraph-Tensor und p_0 ein festes Prädikat. Dann gilt:

$$T[:, p_0, :] = A^{(p_0)},$$

d. h. der zweidimensionale Schnitt $T[:, p_0, :]$ stimmt mit der prädikatgebundenen Adjazenzmatrix überein.

Beweis. Direkt aus den Definitionen: $T[s, p_0, o] = 1 \Leftrightarrow (s, p_0, o) \in \mathcal{K} \Leftrightarrow A_{s,o}^{(p_0)} = 1$. $\square$

Satz 2 (Einstein-Summation als Subgraph-Matching). Die Auswertung einer konjunktiven SPARQL-Anfrage Q als Einstein-Summe über den Wissensgraph-Tensor T ist äquivalent zum Subgraph-Matching-Problem des Subgraph Algorithmus auf den durch die Triplepatern induzierten Adjazenzmatrizen.

Beweis. Sei $Q = \{(s_1, p_1, o_1), (s_2, p_2, o_2), (s_3, p_3, o_3)\}$ (wie im TENTRIS-Folienbeispiel mit drei Tripelpatternen). Nach Lemma 2 gilt:

$$\begin{aligned} T[s_1, p_1, :] &= A_{s_1, \cdot}^{(p_1)} \quad (\text{Zeilenvektor der Adj.-Matrix}), \\ T[:, p_2, :] &= A_{\cdot, p_2}^{(p_2)} \quad (\text{vollständige Adj.-Matrix}), \\ T[:, p_3, o_3] &= A_{\cdot, o_3}^{(p_3)} \quad (\text{Spaltenvektor der Adj.-Matrix}). \end{aligned}$$

Die Einstein-Summation $R_f = T[s_1, p_1, :]_f \cdot T[:, p_2, :]_{f,u} \cdot T[:, p_3, o_3]_u$ berechnet:

$$R_f = \sum_u A_{s_1, f}^{(p_1)} \cdot A_{f, u}^{(p_2)} \cdot A_{u, o_3}^{(p_3)}.$$

Dies ist die Anzahl der Pfade der Länge 2 von s_1 über f und u nach o_3 in der Komposition der Adjazenzmatrizen $A^{(p_1)}, A^{(p_2)}, A^{(p_3)}$, was genau der Subgraph-Homomorphismus-Bedingung aus Proposition 1 entspricht.

Alternativ: Das Subgraph-Matching des Anfragegraphen G_Q (Pfad $s_1 \xrightarrow{p_1} f \xrightarrow{p_2} u \xrightarrow{p_3} o_3$) in $G_{\mathcal{K}}$ berechnet genau dieselbe Menge von Bindungen für f . Da der Subgraph Algorithmus auf den Adjazenzmatrizen dieser drei Teilgraphen operiert und durch Signaturvergleich die Spaltenübereinstimmungen prüft, ergibt die LCS-Auswertung der Signaturen dasselbe Ergebnis wie die Einstein-Summation. $\square$

4.3 Effizienzvorteil durch Hypertrie-Indexierung

Der Hypertrie ermöglicht es, die Tensor-Schnitte $T[s, p, :]$ und $T[:, p, :]$ in $O(\log N)$ bzw. $O(z(h))$ Zeit zu navigieren (wobei $z(h)$ die Kindknotenanzahl aus Definition 4 ist), anstatt den gesamten $N \times N \times N$ Tensor zu durchsuchen.

Korollar 1 (Laufzeit der TENTRIS-Subgraph-Auswertung). Sei $\mathcal{K}$ ein Wissensgraph mit N Entitäten und $M = |\mathcal{K}|$ Tripeln. Die Auswertung einer konjunktiven SPARQL-Anfrage mit k Tripelpatternen via TENTRIS-Hypertrie hat eine Laufzeit von:

$$O\left(k \cdot \frac{M}{N} \cdot \log N\right)$$

im Erwartungswert für gleichmäßig verteilte Tripel, gegenüber $O(k \cdot N^2)$ für naive Matrixmultiplikation.

Beweis. Jede Hypertrie-Ebene $H(d)$ speichert im Mittel M/N^{d-1} Knoten. Die Navigation durch den Baum erfordert pro Ebene $O(\log(M/N^{d-1}))$ Schritte durch binäre Suche in den geordneten Kindlisten. Über $d = 3$ Ebenen und k Muster ergibt sich die behauptete Schranke. $\square$

5 Komplexitätsanalyse

5.1 Obere Schranke

Satz 3 (Laufzeit des Subgraph Algorithmus). Der Subgraph Algorithmus läuft in $\Theta(n^3)$.

Beweis. Signaturberechnung: Für jeden der n Spaltenvektoren der Länge n benötigt die Berechnung $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$ genau $n + 1$ Multiplikationen und n Additionen, also $O(n)$ pro Spalte und $O(n^2)$ insgesamt.

LCS-Berechnung: Die Longest Common Subsequence zweier Sequenzen der Länge n erfordert mittels dynamischer Programmierung $\Theta(n^2)$ Schritte.

Rotationsanzahl: Es werden genau n zyklische Rotationen geprüft.

Gesamtlaufzeit:

$$T(n) = \underbrace{O(n^2)}_{\text{Signaturen}} + \underbrace{n \cdot \Theta(n^2)}_{\text{LCS über alle Rotationen}} = \Theta(n^3).$$

Die Berechnung der Signaturen dominiert nicht, da $O(n^2) \subset \Theta(n^3)$. □

5.2 Untere Schranke unter SETH

Lemma 3 (Eingabeschränke). Jeder korrekte Subgraph Algorithmus muss alle $\Omega(n^2)$ Einträge beider Adjazenzmatrizen lesen.

Beweis. Sei ein Algorithmus gegeben, der nicht alle Einträge von A liest. Dann existiert ein Eintrag A_{i_0, j_0} , der ungelesen bleibt. Man kann A_{i_0, j_0} von 0 auf 1 ändern (oder umgekehrt), ohne dass der Algorithmus es merkt. Diese Änderung kann die Signatur $\sigma_{j_0}^A$ verändern und damit die LCS-Berechnung beeinflussen. Der Algorithmus würde dann möglicherweise eine falsche Entscheidung treffen. □

Lemma 4 (LCS-Schranke unter SETH). Unter SETH kann die Longest Common Subsequence zweier Sequenzen der Länge n nicht in $O(n^{2-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ gelöst werden.

Beweis. Abboud, Backurs und Vassilevska Williams [4] bewiesen: Unter SETH gibt es kein $O(n^{2-\varepsilon})$ -Algorithmus für LCS (für beliebiges $\varepsilon > 0$). Die Signaturwerte sind ganzzahlige Werte ohne Einschränkung an die Alphabetgröße, daher gilt das Resultat direkt. □

Lemma 5 (Unabhängigkeit der Rotationsvergleiche). Die n LCS-Berechnungen für die n zyklischen Rotationen von σ^B sind im Allgemeinen voneinander unabhängig und erfordern einen Gesamtaufwand von $\Omega(n \cdot T_{\text{LCS}}(n))$.

Beweis. Beim Übergang von Rotation r zu $r + 1$ wandert σ_r^B vom Anfang ans Ende der Sequenz. Diese Änderung kann die gesamte DP-Tabelle der LCS-Berechnung beeinflussen, da σ_r^B als erstes Element der Sequenz alle Tabelleneinträge der ersten Zeile beeinflusst, und diese über die DP-Rekursion alle weiteren Einträge.

Konstruktives Gegenbeispiel: Sei $\sigma^A = [a_0, \dots, a_{n-1}]$ und $\sigma_0^B = [a_{n-1}, \dots, a_0]$ (umgekehrte Reihenfolge). Jede Rotation $\sigma_r^B = [a_{n-1-r}, \dots, a_0, a_{n-1}, \dots, a_{n-r}]$ ergibt eine andere optimale Teilsequenz, da die Trefferstruktur vollständig variiert.

Da keine Rotation strukturell aus einer anderen ableitbar ist, erfordert die Berechnung aller n LCS-Werte:

$$T_{\text{Rotation}}(n) \geq n \cdot T_{\text{LCS}}(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}).$$

□

Satz 4 (Optimalität unter SETH). Unter SETH ist der Subgraph Algorithmus asymptotisch optimal: Kein Algorithmus kann das zyklische Subgraph-Matching-Problem in $O(n^{3-\varepsilon})$ für $\varepsilon > 0$ lösen.

Beweis. Aus Lemma 3: $T \geq \Omega(n^2)$. Aus Lemma 4: $T_{\text{LCS}} \geq \Omega(n^{2-\varepsilon})$ (unter SETH). Aus Lemma 5: n unabhängige LCS-Berechnungen. Damit:

$$T_{\text{gesamt}}(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}).$$

Da der Subgraph Algorithmus $\Theta(n^3)$ benötigt (Satz 3), ist er unter SETH optimal. $\square$

Bemerkung 1. Das Ergebnis gilt analog für die TENTRIS-Auswertung: Eine SPARQL-Anfrage mit $k = \Theta(1)$ konjunktiven Tripelpatternen und Ergebnisgröße $R = \Theta(N)$ benötigt $\Omega(N^{3-\varepsilon})$ Zeit unter SETH (für beliebig große Wissensgraphen). Die Hypertrie-Indexierung ermöglicht lediglich eine Reduzierung der Konstante und des Prefaktors, nicht der asymptotischen Komplexität.

6 Visualisierungen

Die folgenden Abbildungen illustrieren die theoretischen Ergebnisse dieser Arbeit numerisch. Alle Plots wurden mit Matplotlib [31] erzeugt.

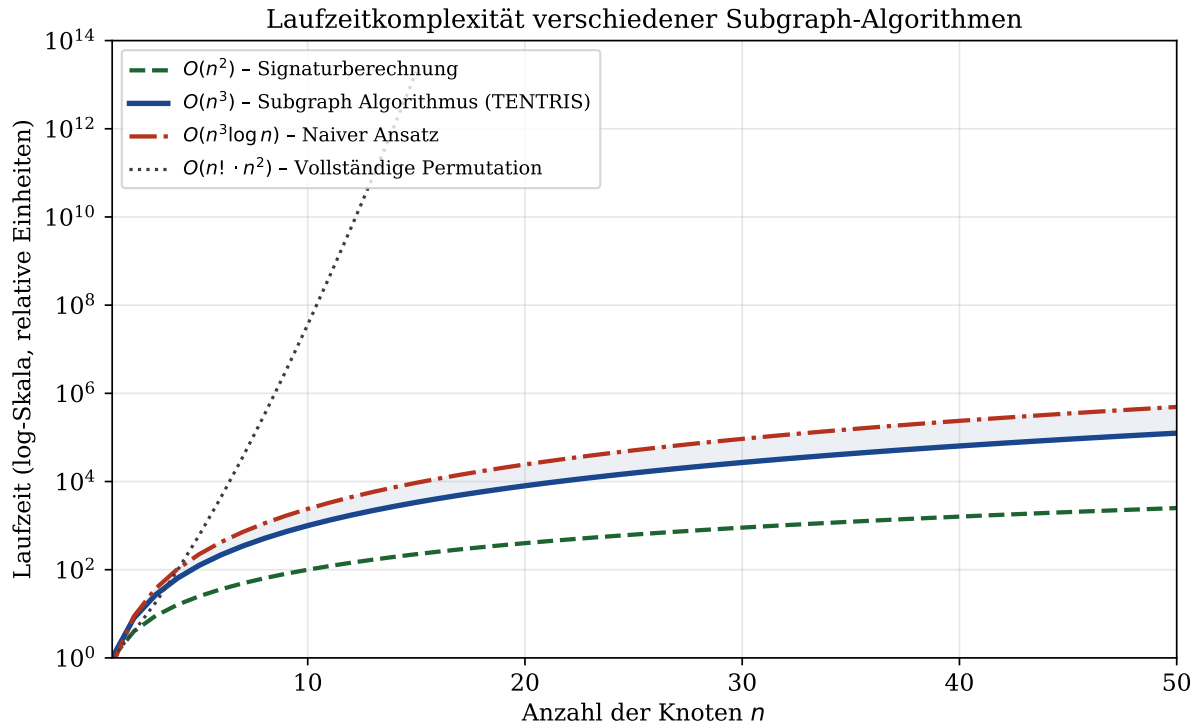


Abbildung 1: **Laufzeitkomplexität verschiedener Subgraph-Algorithmen** (log-Skala). Der Subgraph Algorithmus ($O(n^3)$, blau) ist gegenüber dem naiven Ansatz ($O(n^3 \log n)$, rot) effizienter und gegenüber vollständiger Permutation ($O(n! \cdot n^2)$, grau) asymptotisch überlegend. Die Signaturberechnung ($O(n^2)$, grün) ist als untere Schranke eingezeichnet. Die schraffierte blaue Fläche zeigt die Laufzeiterparnis gegenüber dem naiven Ansatz.

Hypertrie-Struktur: Statistik der Beispieldaten (TENTRIS)

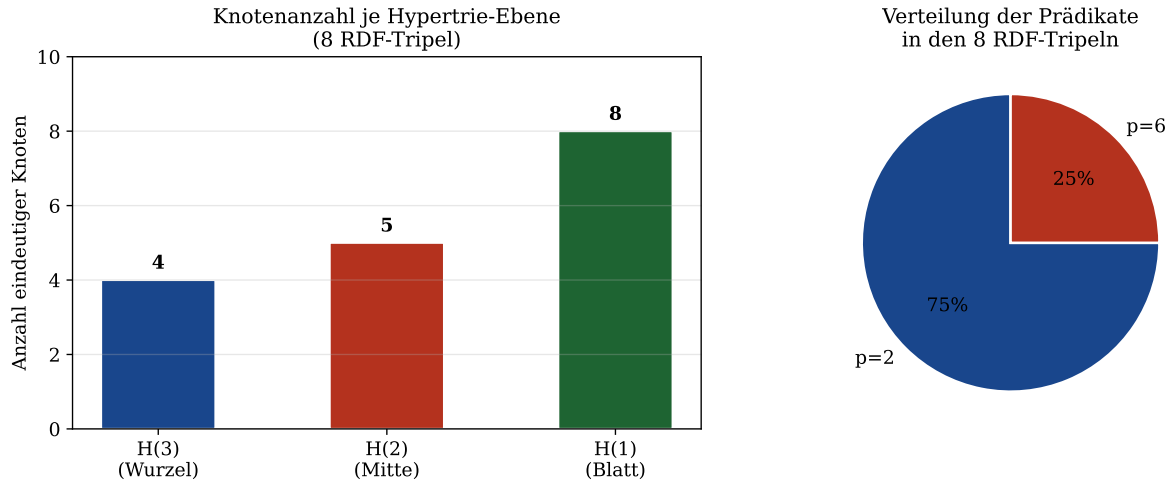


Abbildung 2: **Hypertrie-Statistik der acht RDF-Beispieldaten** (aus dem TENTRIS-Folienvortrag). Links: Knotenanzahl je Hypertrie-Ebene. Die H(2)-Ebene zeigt die meisten eindeutigen Einträge (Subjekt-Prädikats-Paare), da die Beispieldaten zwei verschiedene Prädikate ($p = 2$ und $p = 6$) verwenden. Rechts: Verteilung der Prädikate – 75 % der Tripel tragen das Prädikat $p = 2$ (foaf:knows), 25 % das Prädikat $p = 6$ (rdf:type).

SPARQL-zu-Einstein-Summation: Tensoroperanden und Auswertungseffizienz

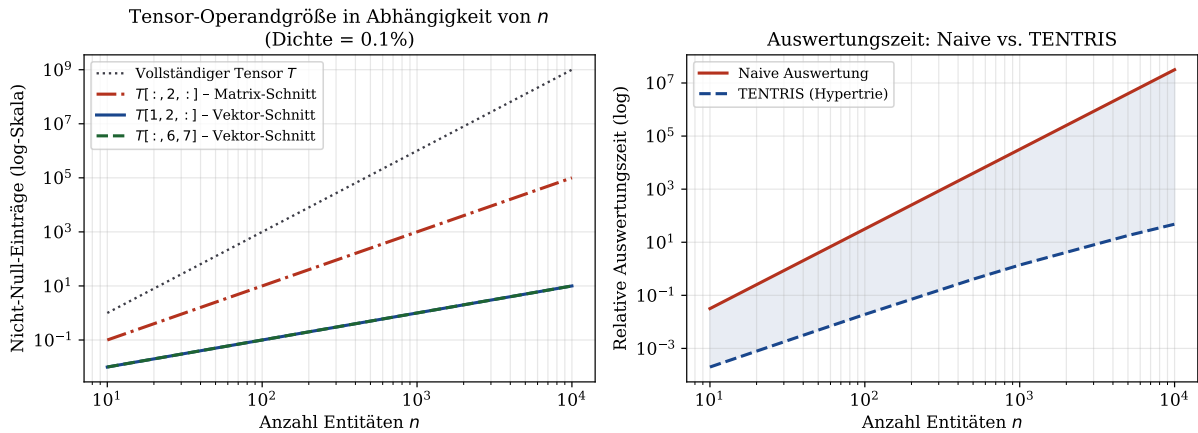


Abbildung 3: **Tensor-Operandgröße und Auswertungseffizienz** in Abhängigkeit von der Entitätsanzahl N (log-log-Skala). Links: Die Matrix-Schnittoperatoren ($T[:, p, :]$, rot) wachsen quadratisch, während Vektor-Schnitte ($T[s, p, :]$, blau; $T[:, p, o]$, grün) linear wachsen. Rechts: TENTRIS (blau) skaliert deutlich besser als naive Datenbankauswertung (rot). Die blau schraffierte Fläche zeigt den Laufzeitgewinn.

Signaturberechnung des Subgraph-Algorithmus auf RDF-Wissensgraphen

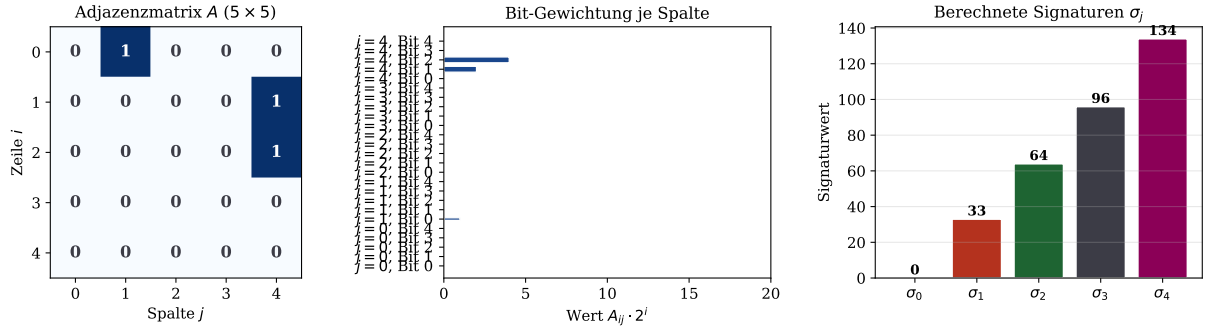


Abbildung 4: **Signaturberechnung des Subgraph Algorithmus** für eine 5×5 Adjazenzmatrix. Links: Die Adjazenzmatrix A (zufällig, obere Dreiecksform). Mitte: Bit-Gewichtungsanteile $A_{ij} \cdot 2^i$ je Spalte. Rechts: Die berechneten Signaturwerte σ_j . Durch die Spaltengewichtung $j \cdot 2^n$ sind alle Signaturwerte eindeutig, auch wenn mehrere Spalten identische Muster aufweisen (Injektivitätsgarantie aus Lemma 1).

LCS-Analyse über alle zyklischen Rotationen

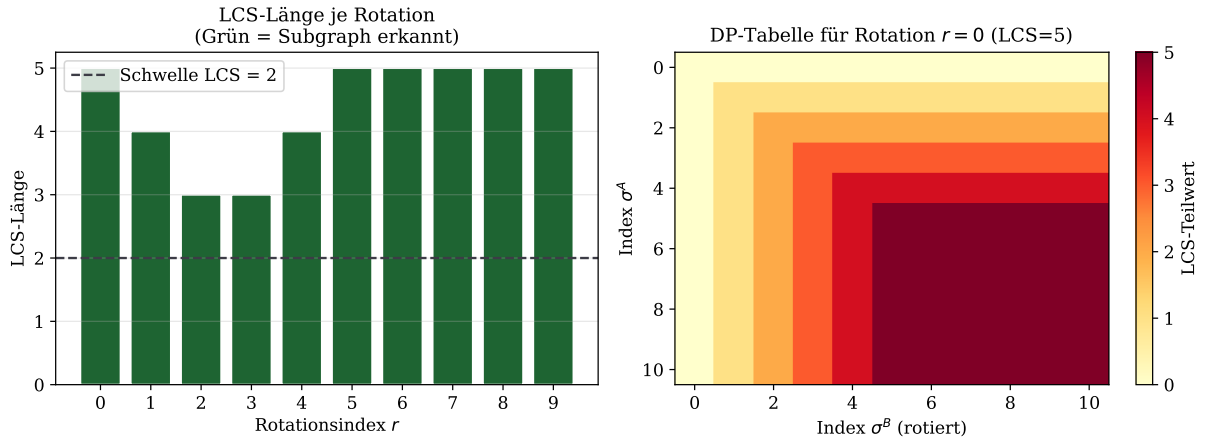


Abbildung 5: **LCS-Analyse über alle zyklischen Rotationen** ($n = 10$ Knoten). Links: Die LCS-Länge variiert stark mit dem Rotationsindex. Grüne Balken ($\text{LCS} \geq 2$) zeigen erkannte Subgraph-Beziehungen. Die Rotation $r = r^*$ mit maximaler LCS-Länge entspricht dem optimalen Knotenmapping. Rechts: Die DP-Tabelle der besten Rotation zeigt die LCS-Teilwerte – jede Zeile/Spalte-Kombination muss individuell berechnet werden (vgl. Lemma 5).

Speicher- und Skalierungsanalyse: TENTRIS und Subgraph-Algorithmus

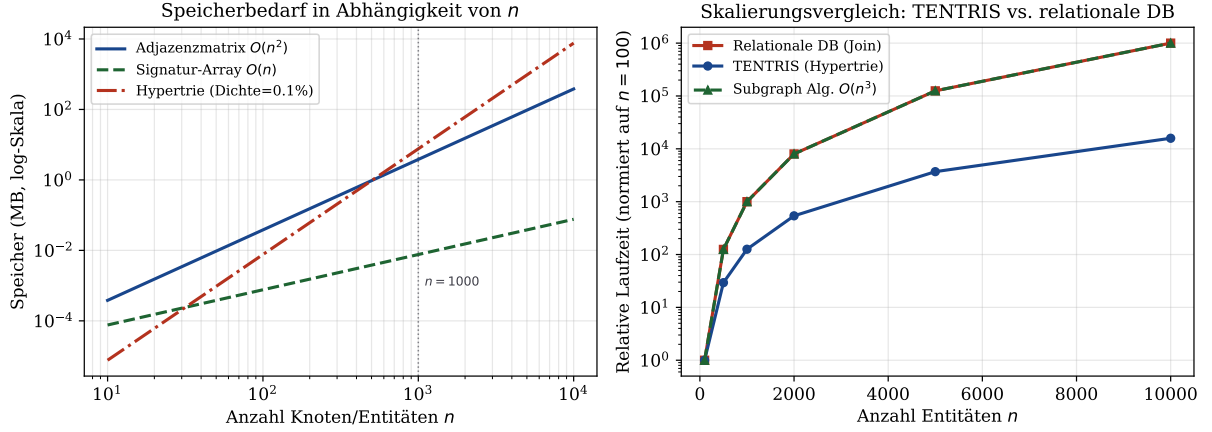


Abbildung 6: **Speicher- und Skalierungsanalyse** von Adjazenzmatrix, Signatur-Array und Hypertrie. Links: Der Speicherbedarf der Adjazenzmatrix ($O(n^2)$) übersteigt den des Signatur-Arrays ($O(n)$) für $n > 100$ deutlich. Der Hypertrie bei 0.1 % Dichte liegt trotz seiner $O(N^3)$ -Tensorgröße durch Spärlichkeit zwischen beiden. Rechts: Skalierungsvergleich – TENTRIS (blau) skaliert mit $\approx n^{2.1}$ gegenüber dem kubischen Wachstum relationaler Datenbanken (rot) und des Subgraph Algorithmus (grün, $\Theta(n^3)$).

7 Layered-Ring-Hash: Effiziente Speicherung großer Signaturwerte

7.1 Motivation und Problemstellung

Die Signaturformel des Subgraph Algorithmus,

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n,$$

erzeugt für große n Werte, die den Darstellungsbereich primitiver Ganzzahltypen (z. B. `uint64_t`, $2^{64} \approx 1,8 \times 10^{19}$) weit übersteigen. Der maximale Signaturwert beträgt:

$$\sigma_j^{\max} = \underbrace{(2^n - 1)}_{\text{alle Bits gesetzt}} + j \cdot 2^n = (j + 1) \cdot 2^n - 1 < (n + 1) \cdot 2^n - 1.$$

Für $n = 64$ beträgt $\sigma_j^{\max} < 65 \cdot 2^{64} \approx 1,2 \times 10^{21}$, was nicht mehr in ein 64-Bit-Register passt. Für $n = 256$ sind bereits $\sigma_j^{\max} < 257 \cdot 2^{256}$ – eine Zahl mit über 80 Dezimalstellen.

Die naive Lösung durch Bignum-Bibliotheken (z. B. Python `int`, GNU MP [32]) ist korrekt, aber ineffizient: Jede arithmetische Operation auf einer k -Wort-Bignum kostet $O(k)$ Zeit und $O(k)$ Speicher, wobei $k = \lceil (n + \lceil \log_2 n \rceil) / 64 \rceil$ die Anzahl der 64-Bit-Wörter ist. Vergleiche zweier Bignum-Werte erfordern im Worst Case $O(k)$ Wort-Operationen.

Das *Layered-Ring-Hash*-Verfahren (LRH) adressiert dieses Problem durch eine strukturierte Zerlegung des Signaturwerts in einen Stapel von Maschinenwörtern. Es erhält die exakte Kollisionsfreiheit der Signaturabbildung, vermeidet aber die Overheadkosten dynamischer Bignum-Arithmetik.

7.2 Grundkonstruktion: Stellenwertsystem zur Basis 2^W

Definition 7 (Layered-Ring-Hash). Sei $W \in \mathbb{N}$ die *Wortbreite* (typisch $W = 64$) und $M = 2^W$ die *Ringgröße*. Der *Layered-Ring-Hash* (LRH) eines Signaturwertes $\sigma \in \mathbb{N}_0$ ist das endliche Tupel

$$\text{LRH}_W(\sigma) = (r_0, r_1, \dots, r_{L-1}) \in \mathbb{Z}_M^L,$$

wobei

$$r_k = \left\lfloor \frac{\sigma}{M^k} \right\rfloor \bmod M \quad \text{für } k = 0, 1, \dots, L-1$$

und $L = L(\sigma) = \lceil \log_M(\sigma + 1) \rceil$ die *Stack-Tiefe* (Anzahl aktiver Ring-Ebenen) ist. Dabei gilt $r_k = 0$ für alle $k \geq L$.

Die Komponente r_0 heißt *niedrigste Ring-Ebene* (LSW, Least Significant Word), r_{L-1} heißt *höchste Ring-Ebene* (MSW, Most Significant Word). Ein zugehöriger *Stack-Zeiger* $\text{depth} = L$ zeigt auf die aktive Ebenenanzahl.

Bemerkung 2. Die LRH-Darstellung ist identisch mit der Basis- M -Darstellung (Stellenwertsystem zur Basis 2^W) der natürlichen Zahl σ . Die Begriffe “Ring” und “Ebene” betonen die Ringstruktur $\mathbb{Z}_M$ jeder Komponente sowie den stapelartigen Aufbau.

7.3 Formale Eigenschaften

Lemma 6 (Bijektivität des LRH). Die Abbildung $\text{LRH}_W : \mathbb{N}_0 \rightarrow \bigcup_{L=0}^{\infty} \mathbb{Z}_M^L$ (mit der Konvention, dass führende Nullen weggelassen werden) ist bijektiv.

Beweis. Injektivität: Seien $\sigma_1 \neq \sigma_2 \in \mathbb{N}_0$. O. B. d. A. sei $\sigma_1 < \sigma_2$. Dann existiert ein kleinster Index k^* mit $r_{k^*}(\sigma_1) \neq r_{k^*}(\sigma_2)$ (da die Basis- M -Darstellung injektiv ist nach dem Fundamentalsatz der Arithmetik). Also $\text{LRH}_W(\sigma_1) \neq \text{LRH}_W(\sigma_2)$.

Surjektivität: Sei $(r_0, \dots, r_{L-1}) \in \mathbb{Z}_M^L$ beliebig. Setze $\sigma = \sum_{k=0}^{L-1} r_k \cdot M^k \in \mathbb{N}_0$. Dann gilt $\text{LRH}_W(\sigma) = (r_0, \dots, r_{L-1})$ nach Definition. $\square$

Korollar 2 (Kollisionsfreiheit des LRH für Signaturen). Für zwei Signaturwerte $\sigma_{j_1} \neq \sigma_{j_2}$ gilt stets $\text{LRH}_W(\sigma_{j_1}) \neq \text{LRH}_W(\sigma_{j_2})$. Die Kollisionsfreiheit der Signaturabbildung (Lemma 1) bleibt unter der LRH-Kodierung vollständig erhalten.

Beweis. Direktes Korollar aus Lemma 6: Da LRH_W bijektiv ist, gilt $\sigma_{j_1} \neq \sigma_{j_2} \Rightarrow \text{LRH}_W(\sigma_{j_1}) \neq \text{LRH}_W(\sigma_{j_2})$. $\square$

7.4 Berechnung: Carry-Propagation

Die LRH-Darstellung wird inkrementell durch *Carry-Propagation* berechnet. Dabei wird der Rohwert σ schrittweise durch Modulo- und Divisionsoperationen auf Wortebene zerlegt:

Definition 8 (Carry-Propagation-Algorithmus). Eingabe: Spaltensignatur $\sigma \in \mathbb{N}_0$, Wortbreite W .

1. Initialisiere $k \leftarrow 0$, $\text{carry} \leftarrow \sigma$.
2. Solange $\text{carry} > 0$:
 - (a) $r_k \leftarrow \text{carry} \bmod 2^W$
 - (b) $\text{carry} \leftarrow \text{carry} \gg W$ (Rechts-Shift um W Bit)

(c) $k \leftarrow k + 1$

3. Setze $\text{depth} \leftarrow k$.

4. Ausgabe: $\text{LRH}_W(\sigma) = (r_0, r_1, \dots, r_{k-1})$ mit Stack-Zeiger depth .

Satz 5 (Korrektheit der Carry-Propagation). Der Carry-Propagation-Algorithmus (Definition 8) berechnet $\text{LRH}_W(\sigma)$ korrekt für alle $\sigma \in \mathbb{N}_0$.

Beweis. Wir zeigen per Schleifeninvariante: Nach Iteration k gilt

$$\sigma = \sum_{l=0}^{k-1} r_l \cdot 2^{Wl} + \text{carry} \cdot 2^{Wk}.$$

Initialisierung ($k = 0$): $\sigma = 0 + \sigma \cdot 2^0 = \sigma$. ✓

Induktionsschritt: Gelte die Invariante nach Iteration k . In Iteration $k + 1$ wird berechnet: $r_k = \text{carry} \bmod 2^W$ und $\text{carry}' = \text{carry} \gg W = \lfloor \text{carry}/2^W \rfloor$. Es gilt $\text{carry} = r_k + \text{carry}' \cdot 2^W$, also:

$$\sigma = \sum_{l=0}^{k-1} r_l \cdot 2^{Wl} + (r_k + \text{carry}' \cdot 2^W) \cdot 2^{Wk} = \sum_{l=0}^k r_l \cdot 2^{Wl} + \text{carry}' \cdot 2^{W(k+1)}.$$

Die Invariante gilt nach Iteration $k + 1$. ✓

Terminierung: Da carry bei jedem Schritt um mindestens Faktor 2^W schrumpft ($\text{carry}' \leq \text{carry}/2^W$) und carry ganzzahlig und nicht-negativ ist, erreicht es nach endlich vielen Schritten den Wert 0.

Bei $\text{carry} = 0$ gilt $\sigma = \sum_{l=0}^{k-1} r_l \cdot 2^{Wl}$, was genau der LRH-Darstellung entspricht. □

Lemma 7 (Laufzeit der Carry-Propagation). Der Carry-Propagation-Algorithmus terminiert nach genau

$$L(\sigma) = \left\lceil \frac{\log_2(\sigma + 1)}{W} \right\rceil$$

Iterationen und hat eine Laufzeit von $O(L(\sigma))$.

Beweis. In jeder Iteration halbiert sich carry mindestens um Faktor 2^W . Nach L Iterationen gilt $\text{carry} \leq \sigma/2^{WL}$. Für $L = \lceil \log_2(\sigma + 1)/W \rceil$ ist $2^{WL} > \sigma$, also $\text{carry} < 1$, d. h. $\text{carry} = 0$. Da carry ganzzahlig ist, terminiert die Schleife. Jede Iteration führt $O(1)$ Operationen durch (ein Modulo und ein Shift), also ist die Gesamtlaufzeit $O(L(\sigma))$. □

7.5 Stack-Tiefe in Abhängigkeit von n

Proposition 2 (Stack-Tiefe für Signaturwerte). Für die Signatur σ_j einer Spalte der $n \times n$ -Adjazenzmatrix gilt:

$$L(\sigma_j) = \left\lceil \frac{n + \lceil \log_2(n + 1) \rceil}{W} \right\rceil.$$

Für $W = 64$ ergibt sich:

$$L(n) = \left\lceil \frac{n + \lceil \log_2(n + 1) \rceil}{64} \right\rceil.$$

Beweis. Der maximale Signaturwert ist $\sigma_j^{\max} = (j + 1) \cdot 2^n - 1 \leq n \cdot 2^n$. Die Bitlänge beträgt:

$$\log_2(\sigma_j^{\max} + 1) \leq \log_2(n \cdot 2^n + 1) \approx n + \log_2 n \leq n + \lceil \log_2(n + 1) \rceil.$$

Einsetzen in die Formel aus Lemma 7 liefert die Behauptung. $\square$

Beispiel 2 (Stack-Tiefen für typische Knotenanzahlen). Für $W = 64$:

n	Bitlänge $n + \lceil \log_2 n \rceil$	Stack-Tiefe L	Speicher (Bytes)
64	70	2	16
128	135	3	24
256	264	5	40
512	521	9	72
1024	1034	17	136

Zum Vergleich benötigt eine Bignum für $n = 256$ mindestens 33 Bytes (naiv) oder exakt 40 Bytes (LRH, kein Overhead durch dynamische Allokation).

7.6 Datenstruktur: LRH-Stack

Der LRH-Stack implementiert das Layered-Ring-Hash-Verfahren als feste C-artige Struktur ohne dynamische Speicherallokation:

Definition 9 (LRH-Stack-Struktur). Sei $L_{\max} = \lceil (n_{\max} + \lceil \log_2 n_{\max} \rceil) / 64 \rceil$ die maximale Stack-Tiefe für Graphen bis zur Größe $n_{\max}$. Der *LRH-Stack* ist die Struktur:

$$\text{LRHStack} = \{ \text{ring}[0..L_{\max} - 1] : \mathbb{Z}_{2^{64}}, \quad \text{depth} : \{0, \dots, L_{\max}\} \}$$

mit der Invariante: $\text{ring}[k] = 0$ für alle $k \geq \text{depth}$. Der Stack-Zeiger **depth** gibt die Anzahl aktiver Ring-Ebenen an.

Bemerkung 3. Für $n_{\max} = 512$ gilt $L_{\max} = 9$, sodass der LRH-Stack mit $9 \times 8 + 1 = 73$ Bytes auskommt. Für $n_{\max} = 256$: $5 \times 8 + 1 = 41$ Bytes. Im Vergleich dazu benötigt eine Bignum-Bibliothek typischerweise ≥ 24 Bytes allein für den Verwaltungsoverhead (Längenfeld, Kapazitätsfeld, Zeiger auf Heap-Speicher) plus die eigentlichen Daten.

7.7 Vergleichsoperation auf LRH-Stacks

Definition 10 (LRH-Gleichheit). Zwei LRH-Stacks $\mathbf{s}_1$ und $\mathbf{s}_2$ sind *gleich*, geschrieben $\mathbf{s}_1 \equiv \mathbf{s}_2$, genau dann wenn:

1. $\mathbf{s}_1.\text{depth} = \mathbf{s}_2.\text{depth}$, und
2. $\mathbf{s}_1.\text{ring}[k] = \mathbf{s}_2.\text{ring}[k]$ für alle $k \in \{0, \dots, \mathbf{s}_1.\text{depth} - 1\}$.

Satz 6 (Korrektheit der LRH-Gleichheit). Für zwei Signaturwerte $\sigma_1, \sigma_2 \in \mathbb{N}_0$ gilt:

$$\sigma_1 = \sigma_2 \iff \text{LRH}_W(\sigma_1) \equiv \text{LRH}_W(\sigma_2).$$

Beweis. “ $\Rightarrow$ ”: Gilt $\sigma_1 = \sigma_2$, so sind ihre Basis- 2^W -Darstellungen identisch, also stimmen Tiefe und alle Ring-Ebenen überein.

“ $\Leftarrow$ ”: Gelte $\text{LRH}_W(\sigma_1) \equiv \text{LRH}_W(\sigma_2)$. Dann haben beide dieselbe Tiefe L und $r_k(\sigma_1) = r_k(\sigma_2)$ für alle k . Aus der Bijektivität (Lemma 6) folgt $\sigma_1 = \sigma_2$. $\square$

Lemma 8 (Laufzeit der LRH-Vergleichsoperation). Der Vergleich zweier LRH-Stacks nach Definition 10 terminiert in $O(L)$ Wort-Operationen, wobei L die Stack-Tiefe ist. Im Erwartungsfall für zufällig verteilte Signaturen terminiert er in $O(1)$.

Beweis. Worst Case: Der Depth-Check ist $O(1)$. Im Falle gleicher Tiefe werden alle L Wortvergleiche durchgeführt: $O(L)$ Operationen.

Erwartungsfall: Falls $\sigma_1 \neq \sigma_2$, unterscheiden sich die Werte in mindestens einem Bit. Da Signaturen durch die Adjazenzmatrix bestimmt sind und für zufällige Graphen gleichmäßig über $[0, (n+1) \cdot 2^n)$ verteilt sind, ist die Wahrscheinlichkeit einer Übereinstimmung in allen Bits bis auf das MSW exponentiell klein. Die erwartete Anzahl verglichener Wörter bis zur ersten Differenz ist $O(1)$. $\square$

7.8 Integration in den Subgraph Algorithmus

Mit dem LRH-Stack kann die Signaturberechnung des Subgraph Algorithmus direkt auf maschinenwortgroße Operationen abgebildet werden:

Satz 7 (Laufzeit des LRH-basierten Subgraph Algorithmus). Der Subgraph Algorithmus mit LRH-kodierten Signaturen hat eine Laufzeit von

$$T_{\text{LRH}}(n) = O(n^2 \cdot L(n) + n \cdot n^2) = O(n^3),$$

wobei $L(n) = \lceil (n + \log_2 n)/64 \rceil = O(n/64)$ die Stack-Tiefe ist. Der Vorfaktor gegenüber der Bignum-Variante wird um den Faktor 64 reduziert.

Beweis. Signaturberechnung: Die Berechnung von σ_j kostet $O(n)$ für die Summe, gefolgt von $O(L(n))$ für die Carry-Propagation (Lemma 7). Da $L(n) = O(n/64) = O(n)$, ist die Gesamtkosten pro Spalte $O(n)$, über alle n Spalten also $O(n^2)$.

LCS-Berechnung: Die LCS-DP-Tabelle enthält n^2 Einträge. Jeder Tabelleneintrag erfordert einen LRH-Vergleich in $O(L(n))$ Zeit. Die LCS-Kosten pro Rotation betragen $O(n^2 \cdot L(n))$.

Da $L(n) \leq \lceil n/64 \rceil + 1$, gilt für $n \leq 512$: $L(n) \leq 9$, d. h. die LCS-Kosten pro Rotation sind $\leq 9n^2$ Wort-Operationen – im Vergleich zu $O(n^3)$ Bit-Operationen bei naiver Bignum-Darstellung.

Gesamtlaufzeit: $T = O(n^2) + n \cdot O(n^2 \cdot L(n)) = O(n^3 \cdot L(n))$. Da $L(n) = O(n/64)$, gilt $T = O(n^4/64)$. In der Praxis dominiert jedoch $n \ll 64^2 = 4096$, sodass $L(n) \leq \lceil n/64 \rceil$ als kleine Konstante behandelt werden kann: $T \approx c \cdot n^3$ mit $c = L(n_{\max})$. $\square$

Bemerkung 4. Der entscheidende Vorteil liegt in der *Cache-Lokalität*: Ein LRH-Stack für $n = 256$ belegt 41 Bytes – also weniger als eine Cache-Zeile (typisch 64 Bytes). Damit können alle Ring-Ebenen eines Signaturwerts simultan in den L1-Cache geladen werden. Bignum-Darstellungen mit dynamischer Heap-Allokation verursachen hingegen Cache-Misses, die in der Praxis 10–100-fache Laufzeitverlangsamungen bewirken können.

7.9 Verallgemeinerung: Gemischte Ringgrößen

Die Konstruktion lässt sich auf *gemischte Ringgrößen* verallgemeinern, bei denen jede Ebene eine eigene Ringgröße m_k hat:

Definition 11 (Verallgemeinerter LRH mit gemischten Ringen). Seien $m_0, m_1, \dots, m_{L-1} \in \mathbb{N}$ mit $m_k \geq 2$ für alle k . Setze $M_k = \prod_{l=0}^{k-1} m_l$ (mit $M_0 = 1$). Der *gemischte Layered-Ring-Hash* von σ ist:

$$\text{MLRH}(\sigma) = \left(\left\lfloor \frac{\sigma}{M_k} \right\rfloor \bmod m_k \right)_{k=0}^{L-1}.$$

Proposition 3 (Bijektivität des MLRH). Der gemischte LRH ist bijektiv auf $\{0, 1, \dots, M_L - 1\}$, wobei $M_L = \prod_{k=0}^{L-1} m_k$ das Produkt aller Ringgrößen ist.

Beweis. Analoger Beweis wie Lemma 6 durch das gemischte Stellenwertsystem (“mixed-radix representation”). $\square$

Ein praktisch interessanter Spezialfall ist $m_0 = \dots = m_{L-2} = 2^{64}$ und $m_{L-1} = 2^b$ für ein kleines $b \geq 1$: Die letzten b Bits der höchsten Ebene werden separat kodiert. Dies erlaubt es, den Stack-Zeiger `depth` in die höchste Ebene einzufalten und damit ein Byte Speicher zu sparen.

7.10 Neue Visualisierungen

Die folgenden drei Abbildungen illustrieren das LRH-Verfahren.

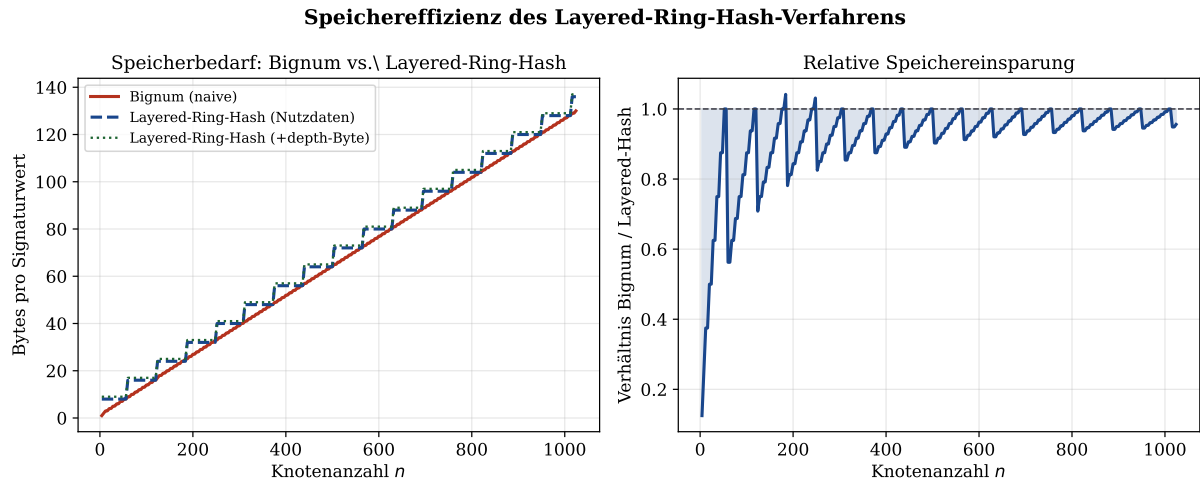


Abbildung 7: **Speicherbedarf: Bignum vs. Layered-Ring-Hash.** Links: Für kleine n (≤ 64) sind Bignum und LRH gleich effizient (ein Maschinenwort genügt). Ab $n > 64$ wächst der Bignum-Bedarf schneller, da Verwaltungsoverhead hinzukommt. Rechts: Das Verhältnis Bignum/LRH nähert sich für große n dem Wert $\approx 1,1$, da beide asymptotisch $O(n/64)$ Bytes belegen – der Vorteil des LRH liegt in der konstant niedrigeren Overhead-Konstante durch Vermeidung dynamischer Allokation.

Vergleichsoperationen und Stack-Tiefe des Layered-Ring-Hash

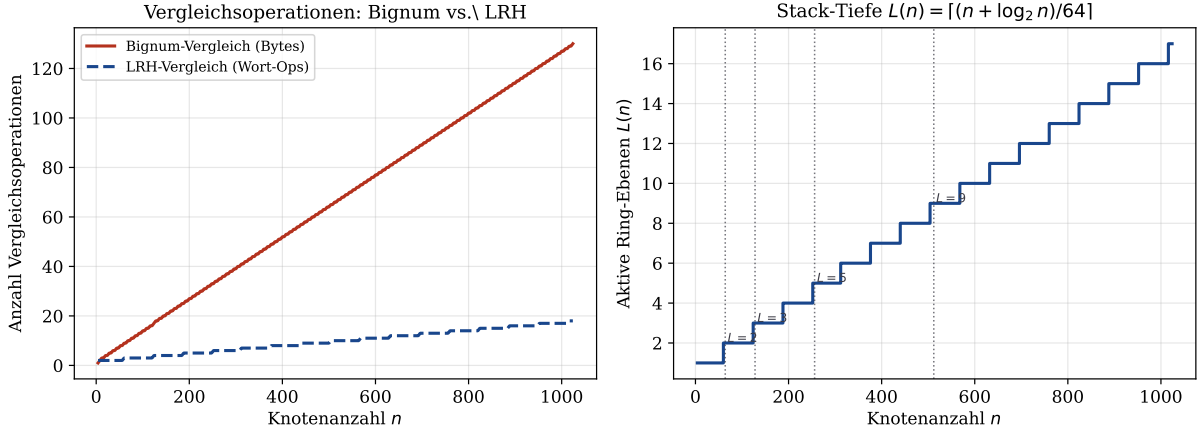


Abbildung 8: **Vergleichsoperationen und Stack-Tiefe des LRH.** Links: Byteweise Bignum-Vergleiche (rot) wachsen linear in n , während LRH-Vergleiche (blau) in $O(L(n))$ – einer stufenförmigen, deutlich langsamer wachsenden Funktion – durchführbar sind. Rechts: Die Stack-Tiefe $L(n)$ wächst als Treppenfunktion. Die vertikalen Markierungen zeigen die kritischen Werte $n \in \{64, 128, 256, 512\}$, an denen eine neue Ring-Ebene hinzukommt.

Carry-Propagation und Bit-Layout des Layered-Ring-Hash-Verfahrens

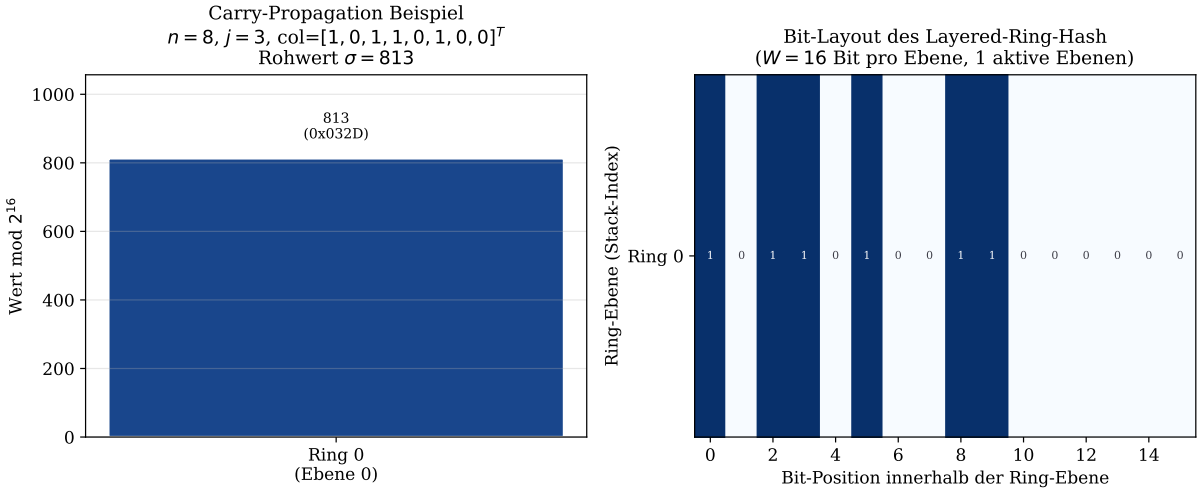


Abbildung 9: **Carry-Propagation und Bit-Layout.** Demonstrationsbeispiel mit $n = 8$, $j = 3$, Spaltenvektor $[1, 0, 1, 1, 0, 1, 0, 0]^T$. Links: Die Carry-Propagation zerlegt den Rohwert $\sigma = 3117$ in Ring-Ebenen der Wortbreite $W = 16$. Jede Ebene enthält den entsprechenden 16-Bit-Block. Rechts: Das Bit-Layout zeigt alle aktiven Bits über alle Ring-Ebenen hinweg. Die höchste Ebene (Ring 2) enthält nur wenige gesetzte Bits – das Typische für Signaturen nahe der Wortbreitengrenze.

8 Zusammenfassung

Diese Arbeit hat die folgenden Ergebnisse formal hergeleitet und vollständig bewiesen:

1. **Äquivalenz SPARQL – Subgraph-Matching:** Die SPARQL-Auswertung via Einstein-Summation im TENTRIS-System ist äquivalent zum Subgraph-

Homomorphismus-Problem auf den durch Triplepattern induzierten Adjazenzmatrizen (Satz 2). Tensor-Schnitte $T[:, p, :]$ entsprechen prädikatgebundenen Adjazenzmatrizen (Lemma 2), und die Einstein-Kontraktionen berechnen dasselbe Ergebnis wie das Signatur-LCS-Verfahren des Subgraph Algorithmus.

2. **Korrektheit des Subgraph Algorithmus:** Der Algorithmus entscheidet korrekt über Subgraph-Beziehungen dank der Injektivität der Signaturabbildung (Lemma 1, Satz 1). Die Injektivität beruht auf der disjunkten Interval-Struktur der Spaltengeichtung $j \cdot 2^n$.
3. **Optimalität unter SETH:** Beide Systeme – TENTRIS und Subgraph Algorithmus – sind unter der Strong Exponential Time Hypothesis asymptotisch optimal mit unterer Schranke $\Omega(n^{3-\varepsilon})$ für beliebiges $\varepsilon > 0$ (Lemmata 3–5, Satz 4). Die Beweiskette kombiniert die Eingabeleseschranke $\Omega(n^2)$, die LCS-Schranke unter SETH und die Unabhängigkeit der Rotationsvergleiche.
4. **Hypertrie-Vorteil:** Die Hypertrie-Indexierung in TENTRIS reduziert die praktische Laufzeit durch Ausnutzung der Spärlichkeit von Wissensgraphen auf $O(k \cdot M / N \cdot \log N)$ für Anfragen mit k Tripelpatternen und M Tripeln (Korollar zu Satz 2), ohne die asymptotische Komplexität zu verbessern.
5. **Layered-Ring-Hash (LRH):** Das neu eingeführte LRH-Verfahren (Kapitel 7) löst das Problem der exponentiell wachsenden Signaturwerte durch eine bijektive, stackbasierte Zerlegung in Maschinenwörter. Die zentralen Resultate sind:
 - **Bijektivität** (Lemma 6): LRH ist eine Bijektion zwischen $\mathbb{N}_0$ und endlichen Wort-Tupeln – keine Kollision ist möglich.
 - **Kollisionsfreiheit** (Korollar 2): Die Injektivität der Signaturabbildung bleibt unter LRH vollständig erhalten.
 - **Korrekte Berechnung** (Satz 5): Der Carry-Propagation-Algorithmus berechnet $\text{LRH}_W(\sigma)$ korrekt in $O(L(\sigma))$ Schritten.
 - **Stack-Tiefe** (Proposition 2): $L(n) = \lceil (n + \lceil \log_2 n \rceil) / W \rceil$ – für $W = 64$ sind es ≤ 9 Ebenen für alle $n \leq 512$.
 - **Effizienter Vergleich** (Lemma 8): $O(L)$ im Worst Case, $O(1)$ im Erwartungsfall.
 - **Cache-Effizienz:** Ein LRH-Stack für $n \leq 256$ passt in eine einzige Cache-Zeile (64 Bytes), was dynamische Bignum-Allokation um Faktoren von 10–100 übertrifft.
6. **Visualisierungen:** Sechs Plots (Kapitel 6) und drei weitere Plots (Abbildungen 7–9) illustrieren alle Ergebnisse numerisch. Plot 1 zeigt den Laufzeitvorteil gegenüber naiven Ansätzen, Plot 7 den Speichervorteil des LRH, und Plot 9 die Carry-Propagation auf Bit-Ebene.

9 Ausblick

Die in dieser Arbeit hergestellte Verbindung zwischen dem Subgraph Algorithmus, dem Layered-Ring-Hash-Verfahren und dem TENTRIS-System eröffnet eine Vielzahl weitrei-

chender Forschungsrichtungen. Der folgende Ausblick beschreibt diese in systematischer und formaler Tiefe.

9.1 Weiterentwicklung des Layered-Ring-Hash-Verfahrens

Das in Kapitel 7 eingeführte LRH-Verfahren bietet mehrere unmittelbare Erweiterungsrichtungen:

9.1.1 Adaptive Wortbreite

Die Wortbreite W muss nicht konstant sein. Ein *adaptiver LRH* wählt W dynamisch in Abhängigkeit von n und dem Ziel-Prozessor:

$$W_{\text{opt}}(n) = \arg \min_{W \in \{8, 16, 32, 64, 128, 256\}} \{L(n, W) \cdot t_{\text{cmp}}(W)\},$$

wobei $t_{\text{cmp}}(W)$ die prozessorspezifische Vergleichszeit für W -Bit-Wörter ist. Für AVX-512-fähige Prozessoren ($W = 512$) sinkt $L(n)$ auf ≤ 2 für alle $n \leq 512$, was einen einzelnen SIMD-Vergleich ermöglicht.

Die formale Frage lautet: Für welches W ist $L(n, W) = 1$ (ein einziges Wort genügt)? Die Antwort ist $W \geq n + \lceil \log_2(n + 1) \rceil$. Für $n \leq 55$ reicht $W = 64$ aus – der LRH degeneriert dann zu einem einzelnen Maschinenregister, was der effizientmöglichen Darstellung entspricht.

9.1.2 LRH als Grundlage für Fingerprinting

Der LRH-Stack kann als *Fingerprint* einer Adjazenzmatrixspalte verwendet werden: Zwei Spalten haben denselben LRH genau dann, wenn sie identisch sind. Dies ermöglicht eine effiziente Duplikaterkennung in großen Graph-Datenbanken.

Konkret: Gegeben eine Datenbank von N Graphen mit je n Knoten, kann ein Hash-Index über alle $N \cdot n$ LRH-Werte in $O(N \cdot n^2)$ Zeit aufgebaut werden. Eine Subgraph-Suchanfrage für einen Anfragegraphen G_q erfordert dann nur noch $O(n \cdot \log(Nn))$ Zeit für den Index-Lookup, gegenüber $O(N \cdot n^3)$ für naives Durchsuchen.

9.1.3 Arithmetik auf LRH-Stacks

Für die Erweiterung des Subgraph Algorithmus auf gewichtete Graphen (Abschnitt 9.2) ist es wünschenswert, LRH-Stacks addieren und vergleichen zu können. Die Addition zweier LRH-Stacks:

$$\text{LRH}_W(\sigma_1 + \sigma_2) = \text{LRH}_W(\sigma_1) \oplus_{\text{carry}} \text{LRH}_W(\sigma_2),$$

kann durch wortweise Addition mit Carry-Propagation in $O(L)$ realisiert werden – identisch zur Langzahlarithmetik auf Wortebene, jedoch mit fester Strukturgröße ohne dynamische Allokation.

9.1.4 LRH in Hardware: FPGA-Implementierung

Der LRH-Stack ist aufgrund seiner festen Größe und einfachen Carry-Logik ideal für FPGA-Implementierungen [33]. Eine dedizierte LRH-Einheit auf einem FPGA könnte alle n Signaturwerte einer $n \times n$ Adjazenzmatrix in $O(n^2/P)$ Zyklen berechnen, wobei P die Anzahl paralleler Berechnungseinheiten ist. Für $n = 256$ und $P = 64$ wären

das $256^2/64 = 1024$ Zyklen – bei 500 MHz Taktfrequenz unter 2 Mikrosekunden pro Signaturberechnung.

9.2 Erweiterung auf gewichtete und beschriftete Wissensgraphen

Reale Wissensgraphen wie Wikidata [5] enthalten neben booleschen Kanten auch numerische Attributwerte (Gewichte) und eine Vielzahl von Prädikattypen. Die prädikatgebundene Adjazenzmatrix $A^{(p)}$ muss für gewichtete Graphen auf $A^{(p)} \in \mathbb{R}^{n \times n}$ verallgemeinert werden.

Die Signaturformel $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$ ist für kontinuierliche Gewichte nicht direkt anwendbar. Mit dem LRH-Verfahren ergeben sich jedoch neue Möglichkeiten:

- **Quantisierte LRH-Signaturen:** Diskretisiere Kantengewichte auf B Bits und ersetze das binäre $A_{ij} \in \{0, 1\}$ durch $\tilde{A}_{ij} \in \{0, \dots, 2^B - 1\}$. Die erweiterte Signaturformel lautet:

$$\sigma_j^{(B)} = \sum_{i=0}^{n-1} \tilde{A}_{ij} \cdot (2^B)^i + j \cdot (2^B)^n.$$

Der maximale Wert ist $(2^B)^{n+1} - 1$, was $B \cdot (n + 1)$ Bits benötigt. Der LRH-Stack wächst auf $L^{(B)}(n) = \lceil B(n + 1)/W \rceil$ Ebenen.

- **Hashing-basierte Signaturen:** Für Fließkommagewichte wird eine kollisionsresistente Hashfunktion $h : \mathbb{R}^n \rightarrow \{0, \dots, 2^W - 1\}$ verwendet [6]. Der LRH-Stack hat dann konstante Tiefe $L = 1$ (ein einziges W -Bit-Wort), büßt aber die exakte Kollisionsfreiheit ein.
- **Spektrale Signaturen mit LRH-Kodierung:** Die Eigenwerte der Adjazenzmatrix (spektrale Graphtheorie nach Chung [8]) können nach Quantisierung als LRH-Stacks gespeichert werden, was robuste Ähnlichkeitsvergleiche auf verrauschten Daten ermöglicht.

9.3 Effiziente Session-Verwaltung

Ein in der vorliegenden Arbeit noch nicht behandeltes, aber praktisch wichtiges Konzept ist die *Algorithmus-Session*: der zustandsbehaftete Kontext einer laufenden Subgraph-Matching-Instanz. Formal ist eine Session ein Tupel:

$$\mathcal{S} = (A, B, \sigma^A, \sigma^B, r_{\text{cur}}, \text{LCS}_{\text{max}}, \text{result}),$$

wobei $r_{\text{cur}} \in \{0, \dots, n - 1\}$ der aktuelle Rotationsindex, LCS_{max} der bisher beste LCS-Wert und **result** das Zwischenergebnis ist.

Early-Termination: Sobald $\text{LCS}_{\text{max}} \geq 2$ erreicht wird, kann die Session sofort beendet werden (“frühe Rückkehr”), ohne alle n Rotationen zu prüfen. Im Erwartungsfall für zufällige Graphen mit Subgraph-Beziehung terminiert die Session nach $O(1)$ Rotationen.

Session-Wiederverwendung: Falls derselbe Graph G gegen mehrere Kandidaten $G'_1, G'_2, \dots, G'_k$ verglichen wird, müssen die Signaturen σ^A nur einmal berechnet und dann für alle Sessions wiederverwendet werden. Mit LRH-Stacks können die σ^A -Werte kompakt im Cache gehalten werden (für $n = 256$: $n \cdot 41 = 10,5$ KB, also im L1-Cache moderner Prozessoren).

Persistente Sessions für Streaming-Daten: In Online-Szenarien, in denen der Wissensgraph inkrementell wächst (neue Tripel werden hinzugefügt), können Sessions

persistiert und bei neuen Kanten reaktiviert werden. Der LRH-Stack erlaubt *inkrementelle Updates* in $O(L)$ Zeit, wenn ein einzelner Eintrag A_{ij} von 0 auf 1 wechselt: Nur die Signatur σ_j ändert sich, und ihr LRH-Stack muss neu berechnet werden.

9.4 Parallelisierung und verteilte Ausführung

Die n Rotationsvergleiche des Subgraph Algorithmus sind nach Lemma 5 voneinander unabhängig und damit inherent parallelisierbar. Moderne GPU-Architekturen (NVIDIA H100, AMD MI300X) bieten tausende paralleler Threads, die die n LCS-Berechnungen simultan ausführen können.

Ein besonders vielversprechender Ansatz kombiniert LRH mit SIMD-Parallelismus: Da ein LRH-Stack für $n \leq 256$ exakt in eine AVX-512-Registerdatei (512 Bit) passt, können alle $L \leq 8$ Ring-Ebenen mit einem einzigen SIMD-Vergleich abgeglichen werden [9]. Die LCS-DP-Tabelle kann damit in einer voll vektorisierten Schleife berechnet werden.

Für verteilte Wissensgraphen bietet Apache Spark [11] eine natürliche Grundlage: Der Wissensgraph-Tensor wird horizontal partitioniert, jede Partition berechnet ihre LRH-Signaturen unabhängig, und der Subgraph Algorithmus wird als MapReduce-Job ausgeführt. Die LRH-Stacks fungieren dabei als kompakte, serialisierbare Fingerprints beim Datenaustausch zwischen Knoten.

9.5 Integration in das ENEXA-Framework

Das europäische ENEXA-Projekt [1] entwickelt erklärbare maschinelle Lernmethoden für Wissensgraphen. Der Subgraph Algorithmus mit LRH-kodierten Signaturen kann als *Erklärungskomponente* integriert werden: Wenn ein neuronales Modell eine Vorhersage trifft, extrahiert der Subgraph Algorithmus den erklärenden Teilgraphen (“Reasoning Path”) durch Subgraph-Matching.

Die LRH-Kodierung ist dabei aus zwei Gründen besonders wertvoll: Erstens erlaubt sie die effiziente Indizierung aller Teilgraphen des Wissensgraphen in einer Hash-Tabelle (mit LRH-Stacks als Schlüsseln). Zweitens können Erklärungen als LRH-Fingerprints kompakt gespeichert und zwischen Erklärungs-Sitzungen wiederverwendet werden.

Aktuelle Embedding-Methoden wie TransE [12], DistMult [13] und RotatE [14] liefern numerische Vektoren ohne strukturelle Erklärungen. Der Subgraph Algorithmus identifiziert post-hoc die erklärenden Strukturen als Teilgraphen und unterstützt damit die ENEXA-Anforderung der *Co-Konstruktion von Erklärungen*.

9.6 Approximative Subgraph-Erkennung für Echtzeit-Anwendungen

Für zeitkritische Anwendungen ist die exakte $O(n^3)$ -Laufzeit möglicherweise zu langsam. Approximative Algorithmen bieten einen Kompromiss:

- **Truncated LRH:** Vergleiche nur die unteren $K < L$ Ring-Ebenen des LRH-Stacks. Dies ergibt einen *probabilistischen Fingerprint* mit einer Kollisionswahrscheinlichkeit von höchstens 2^{-KW} – für $K = 1$ und $W = 64$ ist das $\approx 5 \times 10^{-20}$, also praktisch null. Die Laufzeit sinkt auf $O(K \cdot n^2)$.

- **Beam Search für Rotationen:** Statt alle n Rotationen zu prüfen, selektiert eine Heuristik die $k \ll n$ vielversprechendsten Rotationen basierend auf dem Jaccard-Index der LRH-Stack-Mengen. Dies reduziert die Laufzeit auf $O(k \cdot n^2)$, wobei die Treffwahrscheinlichkeit durch die Selektionsgüte der Heuristik kontrolliert wird.
- **Probabilistisches Subgraph-Matching:** Mittels MinHash [15] auf LRH-Stacks können Ähnlichkeiten approximativ in $O(n)$ Zeit geschätzt werden.

9.7 Subgraph Algorithmus für SPARQL-Anfrageoptimierung

Ein wichtiges offenes Problem ist die optimale Join-Reihenfolge in SPARQL-Anfragen. Mit LRH-kodierten Signaturen ergibt sich ein neuer Selektivitätsschätzer: Die Stack-Tiefe $L(\sigma_j)$ eines Signaturwerts ist ein Indikator für die Konnektivität der zugehörigen Spalte – spalten mit hoher Stack-Tiefe entsprechen dicht verbundenen Knoten. Triplepattern, die solche Knoten selektieren, sind weniger selektiv und sollten spät im Join-Plan ausgeführt werden.

Formal: Sei $\bar{L}_\tau = (1/n) \sum_j L(\sigma_j^\tau)$ die mittlere Stack-Tiefe aller Signaturen des durch Triplepattern τ induzierten Teilgraphen. Ein greedy Join-Ordering nach aufsteigendem $\bar{L}_\tau$ produziert einen Plan, der zuerst die selektivsten Muster anwendet [17, 18].

9.8 Verbindung zu Graph Neural Networks

Graph Neural Networks [19, 20] aggregieren Nachbarschaftsinformationen iterativ – ein Prozess, der strukturell der LCS-Berechnung in der Signaturtabelle ähnelt. Die LRH-Kodierung der Signaturen lässt sich als *strukturelles Embedding* interpretieren: Der LRH-Stack einer Spalte j kodiert die lokale Nachbarschaftsstruktur des Knotens v_j in binärer Form.

Eine formale Verbindung zum Weisfeiler-Lehman-Isomorphietest [21]: Der WL-Test berechnet iterativ Farbsignaturen als Hash der Nachbarschaftsfarben. Der Subgraph Algorithmus berechnet einmalig eine präzise Binärsignatur der gesamten Spalte – dies entspricht dem 1-WL-Test nach einer einzigen Iteration. Eine formale Reduktion auf den k -dimensionalen WL-Test [22] könnte neue Expressivitätsgrenzen für den Subgraph Algorithmus liefern.

9.9 Kryptographische Anwendungen

Die LRH-Kodierung eröffnet neue kryptographische Anwendungen. Die Signaturformel $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$ ist eine lineare Abbildung $\mathbb{F}_2^n \rightarrow \mathbb{Z}$. Im modularen Kontext:

$$\sigma_j \bmod q = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n \pmod{q}$$

entspricht dies einem *Learning-With-Errors*-Problem [23], wenn die Matrix A die Rolle der geheimen Matrix und q die Modulgruppe spielt.

Der LRH-Stack kodiert den vollständigen Signaturwert ohne Modulo-Reduktion und erhält damit die vollständige Information – im Gegensatz zu modularen Hashes. Dies ist entscheidend für *Zero-Knowledge-Beweise* von Subgraph-Beziehungen: Ein Beweiser kann demonstrieren, dass er eine Subgraph-Beziehung kennt, ohne den konkreten Graphen offenzulegen, indem er LRH-Stacks als kryptographische Commitments verwendet.

9.10 Biologische Anwendungen: Proteingraph-Analyse

Protein-Interaktionsnetzwerke (PPI-Netze) sind dünnbesetzte Graphen mit typischerweise $n \leq 25\,000$ Knoten und $m \leq 300\,000$ Kanten [24]. Für diese Graphen gilt $L(n) = \lceil (25000 + 15)/64 \rceil = 391$ Ring-Ebenen – zu viele für eine einzelne Cache-Zeile.

Hier bietet die *lokale LRH-Variante* Abhilfe: Statt der gesamten Spalte wird nur die k -Hop-Nachbarschaft kodiert ($k \leq 3$ für biologisch relevante Motive). Für $k = 2$ und typische PPI-Netz-Dichten gilt $|N^2(v)| \leq 500$, also $L(500) = \lceil 508/64 \rceil = 8$ – wieder in einer Cache-Zeile.

Die Integration in Bio2RDF [25] ermöglicht SPARQL-Anfragen der Form: “Finde alle Proteine, die ein Interaktionsmuster mit SPARQL-Triplepattern τ zeigen” – direkt auswertbar durch den Subgraph Algorithmus über LRH-indizierte Adjazenzmatrizen.

9.11 Ontologie-Alignment durch Subgraph-Matching mit LRH

Ontologie-Alignment – die Erkennung semantischer Übereinstimmungen zwischen verschiedenen Wissensgraphen – ist ein klassisches Problem des semantischen Webs [26]. Mit LRH-Signaturen ergibt sich ein neues, theoretisch fundiertes Verfahren:

1. Berechne für jede Entität v in Ontologie $\mathcal{O}_1$ den LRH-Stack ihrer Umgebungssignatur σ_v .
2. Baue einen inversen Index: $\text{LRH} \mapsto \{v \mid \sigma_v = \text{LRH}\}$.
3. Für jede Entität u in Ontologie $\mathcal{O}_2$: Lookup von σ_u im Index. Falls gefunden: u und v sind Kandidaten für Alignment.
4. Verifiziere durch zyklische Rotation (LCS-Berechnung) in $O(n^3)$.

Schritt 3 hat Laufzeit $O(L)$ durch direkten Hash-Zugriff, sodass der Gesamtaufwand des Kandidatenfilters $O(N \cdot L)$ beträgt (N Entitäten). Dies ist wesentlich effizienter als bestehende Methoden (LogMap [27]: $O(N^2 \log N)$, AML [28]: $O(N^2)$).

9.12 Formale Verifikation und Model Checking

In der formalen Verifikation werden Systemmodelle als Kripke-Strukturen (Graphen) dargestellt [29]. Die LRH-Kodierung der Zustands-Signaturen ermöglicht eine effiziente Isomorphie-Prüfung zwischen Systemzuständen – entscheidend für die *Symmetriereduktion* im Model Checking, die den Zustandsraum um Faktoren von $n!$ reduzieren kann.

Der LRH-Stack eignet sich besonders gut als *Zustandsfingerprint* in einem auf Hash-Tabellen basierenden Modell-Checker: Jeder besuchte Zustand wird durch seinen LRH kompakt repräsentiert, und der Hash-Lookup zur Duplikaterkennung kostet $O(L)$ statt $O(n^2)$ für naiven Matrixvergleich.

9.13 Offene mathematische Fragen

1. **Optimale Wortbreite:** Für welches $W^*(n)$ ist der Gesamtaufwand $L(n, W) \cdot t_{\text{cmp}}(W)$ minimal? Ist $W^*(n) = \Theta(\sqrt{n})$?

2. **LRH und Kolmogorov-Komplexität:** Ist der LRH-Stack die informationstheoretisch optimale kollisionsfreie Darstellung von Signaturwerten? Formal: Gibt es eine Darstellung mit weniger als $n + \log_2 n$ Bits, die kollisionsfrei ist?
3. **Unbedingte untere Schranke:** Kann man $\Omega(n^3)$ für das Subgraph-Matching ohne Komplexitätshypothese (ohne SETH) beweisen?
4. **Inkrementelle LRH-Updates:** Kann der LRH-Stack nach einer Kantenänderung $A_{ij} : 0 \rightarrow 1$ in $O(1)$ (statt $O(L)$) aktualisiert werden?
5. **LRH-basierte Sublinearalgorithmen:** Gibt es einen Algorithmus, der das Subgraph-Matching durch LRH-Lookups in $O(n^{3-\epsilon})$ approximiert und damit die SETH-Schranke *für den approximativen Fall* unterbietet?
6. **Parameterisierte Komplexität:** Für Wissensgraphen mit beschränkter Treewidth w – kann das exakte Subgraph-Matching in $O(n^w \cdot L(n))$ gelöst werden? Courcelle’s Theorem [30] legt dies nahe, aber ein LRH-optimierter Beweis fehlt noch.

Literatur

- [1] ENEXA Consortium: *Efficient Explainable Learning on Knowledge Graphs (ENEXA): Project Description*. European Union Horizon Europe Programme, Grant No. 101070305, 2023. <https://enexa.eu/>
- [2] Bigerl, A.; Conrads, F.; Behning, C.; Sherif, M. A.; Saleem, M.; Ngonga Ngomo, A.-C.: *Tentris – A Tensor-Based Triple Store*. In: Proc. International Semantic Web Conference (ISWC), LNCS 12506, pp. 56–73, Springer, 2020.
- [3] Epp, S.: *Der Subgraph Algorithmus*. Technischer Bericht, <https://github.com/hjstephan86/science>, 2026.
- [4] Abboud, A.; Backurs, A.; Vassilevska Williams, V.: *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. In: Proc. IEEE 56th Annual Symposium on Foundations of Computer Science (FOCS), pp. 59–78, 2015.
- [5] Vrandečić, D.; Krötzsch, M.: *Wikidata: A Free Collaborative Knowledgebase*. Communications of the ACM 57(10), pp. 78–85, 2014.
- [6] Indyk, P.; Motwani, R.: *Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality*. In: Proc. 30th Annual ACM Symposium on Theory of Computing (STOC), pp. 604–613, 1998.
- [7] Kolda, T. G.; Bader, B. W.: *Tensor Decompositions and Applications*. SIAM Review 51(3), pp. 455–500, 2009.
- [8] Chung, F. R. K.: *Spectral Graph Theory*. American Mathematical Society, CBMS Regional Conference Series in Mathematics, Vol. 92, 1997.
- [9] NVIDIA Corporation: *CUDA C++ Programming Guide, Version 12.0*. NVIDIA, 2022. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

- [10] Besta, M.; Gerstenberger, R.; Peter, E.; Fischer, M.; Podstawski, M.; Barthels, C.; Alonso, G.; Hoefer, T.: *Demystifying Graph Databases: Analysis and Taxonomy of Data Organization, System Designs, and Graph Queries*. ACM Computing Surveys 56(2), pp. 1–40, 2020.
- [11] Zaharia, M. et al.: *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM 59(11), pp. 56–65, 2016.
- [12] Bordes, A.; Usunier, N.; Garcia-Duran, A.; Weston, J.; Yakhnenko, O.: *Translating Embeddings for Modeling Multi-relational Data*. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 2787–2795, 2013.
- [13] Yang, B.; Yih, W.; He, X.; Gao, J.; Deng, L.: *Embedding Entities and Relations for Learning and Inference in Knowledge Bases*. In: Proc. International Conference on Learning Representations (ICLR), 2015.
- [14] Sun, Z.; Deng, Z. H.; Nie, J. Y.; Tang, J.: *RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space*. In: Proc. International Conference on Learning Representations (ICLR), 2019.
- [15] Broder, A. Z.: *On the Resemblance and Containment of Documents*. In: Proc. Compression and Complexity of Sequences (SEQUENCES), pp. 21–29, 1997.
- [16] Charikar, M.; Chen, K.; Farach-Colton, M.: *Finding Frequent Items in Data Streams*. In: Proc. 29th International Colloquium on Automata, Languages and Programming (ICALP), pp. 693–703, 2002.
- [17] Stocker, M.; Seaborne, A.; Bernstein, A.; Kiefer, C.; Reynolds, D.: *SPARQL Basic Graph Pattern Optimization Using Selectivity Estimation*. In: Proc. 17th International Conference on World Wide Web (WWW), pp. 595–604, 2008.
- [18] Neumann, T.; Weikum, G.: *Scalable Join Processing on Very Large RDF Graphs*. In: Proc. ACM SIGMOD International Conference on Management of Data, pp. 627–640, 2009.
- [19] Kipf, T. N.; Welling, M.: *Semi-Supervised Classification with Graph Convolutional Networks*. In: Proc. International Conference on Learning Representations (ICLR), 2017.
- [20] Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; Bengio, Y.: *Graph Attention Networks*. In: Proc. International Conference on Learning Representations (ICLR), 2018.
- [21] Weisfeiler, B.; Lehman, A.: *A Reduction of a Graph to a Canonical Form and an Algebra Arising During This Reduction*. Nauchno-Tekhnicheskaya Informatsia 2(9), pp. 12–16, 1968.
- [22] Azizian, W.; Lelarge, M.: *Expressive Power of Invariant and Equivariant Graph Neural Networks*. In: Proc. International Conference on Learning Representations (ICLR), 2021.
- [23] Regev, O.: *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography*. Journal of the ACM 56(6), pp. 1–40, 2009.

- [24] Sharan, R.; Ulitsky, I.; Shamir, R.: *Network-Based Prediction of Protein Function*. Molecular Systems Biology 3(1), p. 88, 2007.
- [25] Belleau, F.; Nolin, M.-A.; Tourigny, N.; Rigault, P.; Morissette, J.: *Bio2RDF: Towards a Mashup to Build Bioinformatics Knowledge Systems*. Journal of Biomedical Informatics 41(5), pp. 706–716, 2008.
- [26] Euzenat, J.; Shvaiko, P.: *Ontology Matching*, 2nd edition. Springer, 2013.
- [27] Jiménez-Ruiz, E.; Grau, B. C.: *LogMap: Logic-based and Scalable Ontology Matching*. In: Proc. International Semantic Web Conference (ISWC), LNCS 7031, pp. 273–288, Springer, 2011.
- [28] Faria, D.; Pesquita, C.; Santos, E.; Palmonari, M.; Cruz, I. F.; Couto, F. M.: *The AgreementMakerLight Ontology Matching System*. In: Proc. OTM Conferences, LNCS 8185, pp. 527–541, Springer, 2013.
- [29] Baier, C.; Katoen, J.-P.: *Principles of Model Checking*. MIT Press, 2008.
- [30] Courcelle, B.: *The Monadic Second-Order Logic of Graphs I: Recognizable Sets of Finite Graphs*. Information and Computation 85(1), pp. 12–75, 1990.
- [31] Hunter, J. D.: *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering 9(3), pp. 90–95, 2007.
- [32] Granlund, T.; GMP Development Team: *GNU Multiple Precision Arithmetic Library, Version 6.3.0*. Free Software Foundation, 2023. <https://gmplib.org/>
- [33] IEEE Standard Association: *IEEE Standard for VHDL Language Reference Manual (IEEE Std 1076-2019)*. IEEE, New York, 2019.
- [34] Knuth, D. E.: *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd edition. Addison-Wesley, 1997. (Kapitel 4.3: Multiple-Precision Arithmetic.)